Report on the modifications to `cssp.py`

chronology of the changes and final diagnosis.

https://github.com/jacopobellosi/Solving-the-Subset-Sum-Problem-via-Quantum-Walk-Search

April 24, 2026

1 The starting problem

The paper *"Solving the Subset Sum Problem via Quantum Walk Search"* (Lancellotti et al., IEEE TC, vol. 75, n. 1, 2026) provides a reference `myQLM` implementation in the GitHub repository `paper-codes/2025-TC`. The initial assumption was that this code could be run end-to-end on a classical state-vector simulator (`PyLinalg`) and that the solution would be visibly amplified on the vertex register S_1. Instead, a direct run of `cssp.py` on the small toy instance n=3, k=1, `values`=[1,2,3], `target_sum`=3 (chosen only because it is small enough to be simulated) returned a *flat* probability distribution over the three admissible values. A second run, instrumented with checkpoints at every stage of the circuit, shows that the circuit is compiled and simulated without errors, but that the phase flip applied by the oracle is not converted into amplitude by the diffusion step.

(a) Direct run of `cssp.py`

```
n=3, k=1, values=[1,2,3], target=3
{'nbqbits': 28, 'gate_size': 2547}

Final measurement on S_1:
  0.33333  |1>
  0.33333  |2>
  0.33333  |3>

(runtime: ~11h CPU on a single core)
```

(b) Run with intermediate checkpoints

```
Ckpt 1  Dicke + VBE:
  6 states, p=0.167, ampl=+0.408
  uniform over the 3 subsets

Ckpt 2  Edge superposition (U_U):
  12 states, p=0.083, ampl=+0.289
  uniform over all edges of J(3,1)

Ckpt 3  Post-Oracle:
  12 states, p=0.083,
  ampl=+0.289 on Sum=2
  ampl=-0.289 on Sum=3  <-- phase flip OK

Ckpt 4  Post-Ref A (qpe qubit 0):
  12 states, ampl +/-0.289, signs scrambled

Ckpt 5  Post-Ref B (qpe qubit 0):
  24 states, ampl +/-0.204, signs flat
  no concentration on Sum=3

Ckpt 6  End of iteration:
  mixed state; flat prob on S_1

Final measurement on S_1:
  0.33333  |1>
  0.33333  |2>
  0.33333  |3>
```

Table 1: Output of the original code. The final distribution on S_1 is identical to the initial Dicke-state distribution $|D_1^3\rangle$. The oracle correctly applies a phase flip to the marked states; the approximate reflection scatters the signs instead of rotating them.

No run produces simulator errors: the circuit is built, linked and simulated to the end. The problem is therefore not a low-level implementation bug, but something either algorithmic or "regime-related". The modifications listed in Section 2 were the sequence of attempts to understand where the amplification was lost.

2 The modifications (chronological order)

The chronology below is reconstructed from the git history; commits sharing the same underlying idea are grouped into a single subsection. All final measurements were taken on the same toy instance $J(3,1)$ with target p=3, to keep a homogeneous baseline for the comparison.

Modification 1 – Making Reflections A and B symmetric on the full coin (17 March 2026)

The paper defines the walk operator as a double reflection,

$$U_W \;=\; U_{\mathrm{ref}}(a_{v_j})\,U_{\mathrm{ref}}(a_{v_i}) \;=\; U_U U_U\,U_{\mathrm{ref}}(0)\,U_U U_U\,U_{\mathrm{ref}}(0)\,U_U,$$

i.e. two symmetric reflections around the all-zero coin state. In the original code *both* reflections were implemented asymmetrically: Reflection A (lines 130–140 of `code_2025TC/cssp.py`) controlled the phase flip only on `wstate_ones` via `Z.ctrl(k)`, and Reflection B (lines 143–153) did the analogous thing on `wstate_zeros` only, with a further typo in the second X-sandwich (`for j in range(k)` instead of `range(n-k)`). The snippet below shows Ref B; Ref A had the analogous defect and is fixed in the same way.

```
# --- Before (original) ---
prw.apply(qrw_update.dag(),
          node_s_zeros, node_s_ones,
          node_t_zeros, node_t_ones,
          alpha_zeros, alpha_ones,
          wstate_zeros, wstate_ones)
for j in range(n - k):
    prw.apply(X, wstate_zeros[j])
prw.apply(Z.ctrl(n - k), qpe_s[qw_iter], wstate_zeros)
for j in range(k):          # BUG: should be range(n-k)
    prw.apply(X, wstate_zeros[j])

# --- After (17 March 2026) ---
prw.apply(qrw_update.dag(),
          node_t_ones, node_t_zeros,
          node_s_ones, node_s_zeros,
          alpha_ones, alpha_zeros,
          wstate_ones, wstate_zeros)
for j in range(k):
    prw.apply(X, wstate_ones[j])
for j in range(n - k):
    prw.apply(X, wstate_zeros[j])
prw.apply(Z.ctrl(n), qpe_s[qw_iter],
          wstate_ones, wstate_zeros)    # control on full coin
for j in range(k):
    prw.apply(X, wstate_ones[j])
for j in range(n - k):
    prw.apply(X, wstate_zeros[j])
prw.apply(qrw_update,
          node_t_ones, node_t_zeros,
          node_s_ones, node_s_zeros,
          alpha_ones, alpha_zeros,
          wstate_ones, wstate_zeros)
```

```
n=3, k=1, values=[1,2,3], target=3
{'nbqbits': 28, 'gate_size': 2581}
0.33333  |1>
0.33333  |2>
0.33333  |3>
(runtime: 11h 37min CPU)
```

After the fix, *both* Reflection A and Reflection B match the formula above. The bug is real and had to be fixed for correctness, but on this toy instance it is not what is hiding the amplification: the output distribution is still flat.

Modification 2 – Removal of the α/ω cleanup block (20–23 March 2026)

Inside the `compute()` block of the QPE, after the ℓ_s controlled walk steps, the original code performed a "cleanup" of the ancilla registers α and ω: a series of `copy_register(m).ctrl()` gates followed by `generate(...)`. The original intent was to uncompute the ancillas to preserve reversibility. The issue is that at that point α and ω are *not* in $|0\rangle$: they are in a superposition that is entangled with the visited vertices. Calling `generate` as if they were in $|0\rangle$ does not remove gates, it adds them: the cleanup acts as a non-trivial unitary on an entangled state instead of an identity, corrupting the expected state of α and ω right before the final QFT and breaking the intended uncompute relation. The correct uncomputation is already provided automatically by myQLM's `prw.uncompute()` at the end of the compute block.

```python
# --- Before (original) ---
for j in range(k):
    prw.apply(qregs_init.copy_register(m).ctrl(),
              wstate_ones[j], node_s_ones[j], alpha_ones)
for j in range(n - k):
    prw.apply(qregs_init.copy_register(m).ctrl(),
              wstate_zeros[j], node_s_zeros[j], alpha_zeros)
prw.apply(generate(k, 1),       wstate_ones)
prw.apply(generate(n - k, 1),   wstate_zeros)

# --- After (20--23 March 2026) ---
# Block removed: it was destroying the walk coherence.
# The uncomputation of the QPE ancillas is already handled
# by prw.uncompute() at the end of the compute() block
# (AQASM's standard compute/uncompute mechanism), so the
# manual cleanup is both redundant and wrong.
```

```
n=3, k=1, values=[1,2,3], target=3
{'nbqbits': 28, 'gate_size': 1359}     # -47% gates
0.33333  |1>
0.33333  |2>
0.33333  |3>
```

The gate count drops by about 47%, confirming that the cleanup block was indeed redundant. The output distribution, however, stays the same: the bug was genuine but, on this toy instance, it was not what was hiding the amplification either.

Modification 3 – Using the *low-width* insert instead of the *low-depth* one (22 March 2026)

The paper explicitly describes *two* implementations of the update operator (Sec. III of the paper): *low-width*, which uses fewer ancillas and is sequential, and *low-depth*, which is parallel but needs extra ancillas. From the paper:

> *"We report both a low-width and low-depth implementation of the update operator. ... The low-depth strategy ... improves circuit depth at the cost of additional ancillary qubits."*

In the reference code `cssp.py` the `delete` routine was internally calling `insert_ld(...).dag()`. Empirically, running `delete` back-to-back with `insert` via `insert_ld` failed a reversibility sanity check (the composite was not the identity on a small non-ancilla input), while the same pattern with `insert_lw` passed; a source-level comment in `sliding_sort_array.py` already flagged `insert_ld` as "not properly reversible (garbage in ancillas)". Since the walk operator $U_W = U_U \cdots U_U$ composes many `insert`/`delete` calls, any leakage in the ancillas of a single call propagates and degrades the QPE-based approximate reflection. The fix is therefore to route `delete` through `insert_lw(...).dag()`. The precise source of the failure inside `insert_ld` was not localised: the point below is only that swapping to `insert_lw` is sufficient in practice.

```
# --- Before ---
from .sliding_sort_array import insert_ld
insert = insert_ld                          # leaves garbage in ancillas

# --- After (22 March 2026) ---
from .sliding_sort_array import insert_lw, insert_ld
insert = insert_lw if low_width else insert_ld     # reversible
# delete() in sliding_sort_array.py updated to use
# insert_lw(...).dag() internally
```

```
n=3, k=1, values=[1,2,3], target=3
{'nbqbits': 28, 'gate_size': 1366}
0.33333  |1>
0.33333  |2>
0.33333  |3>
```

The update operator is now reversible, as required by the MNRS theory. This is a structural correctness fix for U_U, but it does not change the measurable dynamics of the small toy instance: the output is still flat.

Modification 4 – Clamping the spectral gap and setting a minimum QPE precision (27 March 2026)

Spectral gap in one line. The spectral gap δ controls how long the quantum walk has to run before a quadratic speed-up appears: the MNRS cost scales as $1/\sqrt{\varepsilon\,\delta}$, with ε the fraction of marked vertices. A small δ means the walk needs many steps to mix, and MNRS wins against brute force; a δ close to 1 means the walk mixes in a handful of steps, so $1/\sqrt{\delta} = O(1)$ and there is no quadratic regime to observe.

ℓ_s in one line. The integer ℓ_s is the number of qubits allocated to the auxiliary QPE register on which Hadamards, controlled walk powers $U_W^{2^j}$ and the final QFT act: it sets the *phase resolution* of the approximate reflection. With ℓ_s qubits the QPE distinguishes 2^{ℓ_s} possible phases. For the approximate reflection to act as $+I$ on the stationary state $|E\rangle$ and as $-I$ on its complement (up to a global phase), ℓ_s must be large enough that the stationary eigenphase of U_W falls into a bin separated from the nearest non-stationary eigenphase. The paper adopts the standard MNRS precision requirement (cf. Magniez–Nayak–Roland–Santha 2011, Thm. 6), which for a walk with spectral gap δ reads $2^{\ell_s} \geq \pi/(2\sqrt{\delta})$, hence

$$\ell_s \;=\; \left\lceil \log_2 \frac{\pi}{2\sqrt{\delta}} \right\rceil .$$

If ℓ_s is too small the QPE "sees" all eigenphases as the same bucket and the block stops acting like a reflection; if ℓ_s is larger than needed, the circuit grows but the behaviour does not improve.

The spectral gap in this formula is the standard one of the transition matrix (equivalently, the normalized-Laplacian gap), and for the Johnson graph $J(n,k)$ is known in closed form:

$$\delta \;=\; \frac{n}{k(n-k)}.$$

This is exact, but the formula is designed for instances with $k(n-k) \geq n$. For the toy instance $(n,k) = (3,1)$ one gets $\delta = 1.5$, which is a formal artifact: a genuine spectral gap of a reversible walk sits in $(0,1]$, and $\delta > 1$ signals that we are outside the regime where the Johnson-graph formula is meant to apply. Plugging this value back into ℓ_s yields $\ell_s = 1$: a single QPE qubit is not enough to resolve the phase of $|E\rangle$ from the other eigenphases of U_W, so the block that is supposed to implement an approximate reflection does not act as a reflection at all. As a defensive patch, δ is clamped to 1 and ℓ_s is forced to at least 3 qubits.

```
# --- Before ---
delta  = n / (k * (n - k))
len_s  = int(np.ceil(np.log2(np.pi / (2 * np.sqrt(delta)))))

# --- After (27 March 2026) ---
# The paper formula is designed for k*(n-k) >= n; for smaller
# instances 1/sqrt(delta) becomes O(1) and the QPE block is
# degenerate. Clamp delta to 1 so that the rest of the code
# does not produce negative len_s.
delta  = min(n / (k * (n - k)), 1.0)
# A single QPE qubit only distinguishes phase 0 from pi,
# which is not enough to approximate a reflection.
len_s  = max(int(np.ceil(np.log2(np.pi / (2 * np.sqrt(delta))))), 3)
```

```
n=3, k=1, values=[1,2,3], target=3
{'nbqbits': 28 -> 30, 'gate_size': 1366}
0.33333  |1>
0.33333  |2>
0.33333  |3>
```

The QPE precision goes from 1 to 3 qubits, but the output is still flat. The clamp is only a numerical safeguard: it does not create a spectral gap that the toy graph does not have.

Modification 5 – Checkpoint debugger for amplitude inspection (17 April 2026)

To find out *where* the amplification is lost, an instrumented copy `cssp_with_checkpoint.py` was written. It simulates the full state at six key points of the circuit (Dicke+VBE, edge superposition, post-oracle, post-QPE, post-diffusion, end of iteration). At every checkpoint it prints probability and complex amplitude of every state with $p > 5 \cdot 10^{-4}$, parsing the raw bitstring into the named registers (Dicke, S_1, S_0, T_1, T_0, α, ω, QPE, Sum).

```
def simulate_and_print_state(prw, n, k, m, len_s, n_qubits_sum, label):
    circ = prw.to_circ(link=[classarith, cuccaro_arith])
    res  = PyLinalg().submit(circ.to_job())
    for sample in res:
        if sample.probability > 5e-4:
            raw = str(sample.state)[1:-1]
            # ... parse raw into dicke / S_1 / S_0 / t_* / a_* / w_* / qpe / sum
                ...
            print(f"Prob: {sample.probability:.5f} "
                  f"| Ampl: {sample.amplitude:+.4f}"
                  f"| S_1:{s1_str} | Sum:{sum_str}")
```

```
Ckpt 2 (Edge Superposition):
  Prob 0.0833 | Ampl +0.2887 | S_1:0 | Sum:2     (non-marked)
  Prob 0.0833 | Ampl +0.2887 | S_1:1 | Sum:3     (marked)   x 8 states

Ckpt 3 (Post-Oracle):
  Prob 0.0833 | Ampl +0.2887 | S_1:0 | Sum:2
  Prob 0.0833 | Ampl -0.2887 | S_1:1 | Sum:3     <-- phase flip OK

Ckpt 5 (Post-Diffusion):
  Prob 0.0417 ... Ampl +/-0.2041 scattered
```

```
  (no concentration on Sum:3)

Ckpt 6 (End of iteration):
  Prob 0.0833 ... Ampl +/-0.2887 signs mixed across S_1=0 and S_1=1
```

This modification does not touch the algorithm, but it was decisive for the diagnosis: it shows empirically that *the oracle works* (the phase flip appears on the marked states at Checkpoint 3) and that the problem appears between Checkpoint 3 and Checkpoint 5, i.e. inside the QPE-based approximate reflection.

Why this matters and what it implies. At Checkpoint 3 the circuit is in exactly the state that a textbook Grover diffusion would now rotate into an amplitude peak: non-marked states have amplitude $+0.289$, marked states have -0.289. Up to a global phase, the approximate reflection $U_{\text{ref}}(E)$ is supposed to act oppositely on the stationary walk state $|E\rangle$ and on its orthogonal complement ($+I$ on one and $-I$ on the other in MNRS; conventions differ by a global sign in some references), producing a net rotation that concentrates probability on the marked subspace. In practice we see the signs become mixed and evenly distributed, i.e. the reflection is acting like a random sign-scrambler rather than a reflection. This is the fingerprint of a QPE run with ℓ_s too small to resolve the phase of $|E\rangle$ from the other eigenphases of U_W, which on $J(3,1)$ is unavoidable because the walk mixes in $O(1)$ steps and $1/\sqrt{\delta} = O(1)$: no further patch inside the QPE block will fix the result on this instance, the regime itself lies outside the one in which MNRS delivers its quadratic speed-up.

3 Why it does not work

3.1 Conclusions

After the modifications in Section 2 the final distribution on S_1 is still exactly $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$. The reason is not a single local bug, but a mismatch between the regime in which MNRS is designed to work and the toy instances chosen for the experiments. The instances $J(3,1)$ and $J(4,2)$ used during debugging were picked as deliberately *small* examples, so that the full state vector could be simulated on a laptop. They are not representative of the instances the paper targets, and their job was only to make the pipeline runnable.

Case 1 – $J(3,1)$ (the toy example used for debugging). Intuitively: on $J(3,1)$ the Johnson graph is just the triangle K_3, three vertices all connected to each other. A walker on a triangle mixes in one step, so there is no "slow direction" in the walk that a QPE with $\ell_s = 1$ qubit can detect. Plugging $(n,k) = (3,1)$ into the paper formula gives $\delta = 3/(1 \cdot 2) = 1.5$ and, from the MNRS cost scaling $1/\sqrt{\varepsilon\,\delta}$ with $\varepsilon = 1/3$, a total of about two "amplification" iterations of a one-step walk: the quantum-walk speed-up collapses to $O(1)$ and the run essentially reduces to a single oracle plus a reflection, i.e. there is no quadratic regime to observe. In practice no tuning inside the QPE block (precision ℓ_s, inverse or direct QFT, scaled powers $U_W^{2^j}$, symmetric Reflection B) can produce a non-uniform distribution on S_1 here: forcing ℓ_s by hand does not bring back a gap that the graph does not have, and the approximate reflection cannot act as the textbook $+I$ on $|E\rangle$ / $-I$ on the complement when only one or two eigenphases can be resolved.

Case 2 – $J(4,2)$. For $(n,k) = (4,2)$ we have $\delta = 4/(2 \cdot 2) = 1$ exactly, a borderline case. The algorithm is formally defined, but there is no spectral separation to exploit: even with $\ell_s = 3$–5 and the scaled powers $U_W^{2^j}$ one gets only a marginal amplification, and with just $\binom{4}{2} = 6$ vertices a direct Grover search on the vertex register is simpler and at least as effective in practice.

Case 3 – Genuine $J(n,k)$ with $\delta < 1$. For instances such as $(5,2)$, $(6,2)$, $(6,3)$ the spectral gap is strictly below 1 and MNRS is in its native regime: with the fixes of Section 2 the algorithm should amplify the solution. The obstacle here is purely computational (see Section 3.2): the

qubit count grows like $\Theta(n \log n)$ and the state-vector simulator cannot follow. In other words, $J(3,1)$ and $J(4,2)$ were chosen because they are simulable, not because they satisfy the theoretical preconditions of the algorithm; the instances where the algorithm does work are out of reach for a laptop simulator.

3.2 Distinct causes

It is important to separate three causes that look like the same symptom on a flat output distribution but are actually independent. The first is a purely *physical* limit of classical simulation: already at $(3,1)$ the circuit uses 28 qubits (Dicke, S_1, S_0, T_1, T_0, α_1, α_0, ω_1, ω_0, QPE and Sum registers, as shown by the `Program qubits` dump in `01_baseline_original_code_flat_output.txt`), and `PyLinalg` is a full state-vector simulator, so every gate touches a vector of $2^{28} \approx 2.7 \cdot 10^8$ complex amplitudes; with about 2500 gates per iteration a single run already lasts several hours on a single core. Growing (n,k) into the regime $\delta < 1$ pushes the qubit count above 35, multiplying the state-vector size by ~ 128 and making the simulation unfeasible. The second cause is a *structural* limit of MNRS on trivial instances: the algorithm requires $\delta < 1$ with some margin to concentrate the measure on the marked state, and $J(3,1)$ simply does not have that margin; the graph is K_3, so its spectral gap is $\Theta(1)$ and there is no slow direction for the QPE to discriminate $|E\rangle$ from the rest of the spectrum. No amount of tuning inside the reflection block can widen the spectral gap of U_W on K_3: the techniques tried in Section 2 (symmetric Ref A/B, 2^j scaling, δ clamp) fix genuine bugs but operate downstream of U_W, whose spectrum is fixed by the graph. The third cause is the set of *genuine bugs* in the code that are independent of the regime: Modification 1 (the asymmetric Ref B) and Modification 2 (the wrong cleanup block) are real bugs and they have been fixed, and in a regime where MNRS has a quadratic window to exploit they would be load-bearing; on $J(3,1)$, however, there is no amplification window to begin with, which is why fixing them is necessary for correctness but invisible on the final distribution.

3.3 Viable alternatives

Scope of the alternatives. The two routes described below are *not* meant to replace MNRS in its native regime: they are pragmatic alternatives for the narrow window in which one wants to run the algorithm *on small instances that still fit a state-vector simulator.* Concretely, the paper-faithful MNRS circuit uses already 28 qubits at $(n,k) = (3,1)$ and grows as $\Theta(n \log n)$, crossing the ≈ 30-qubit ceiling of `PyLinalg` on commodity hardware around $(5,2)$–$(6,3)$. The interesting window is therefore roughly

$$n \lesssim 6\text{–}7, \qquad N = \binom{n}{k} \lesssim 20\text{–}30 \text{ vertices},$$

This is orders of magnitude below the paper's own target ($n \geq 25$ for lower T-count, $n \geq 2^{13}$ for lower T-depth), but that gap is expected: classical state-vector simulation caps at roughly 30 qubits on commodity hardware. Above this threshold the simulator cannot follow and the only meaningful execution is on fault-tolerant quantum hardware as envisioned by the paper; below it, the alternatives below produce a working demo on a laptop without pretending to recover the asymptotic MNRS speed-up.

There are essentially two reasonable ways to move forward. The first is to drop MNRS altogether and run a *direct Grover search* on the vertex register: for $N = \binom{n}{k}$ small (up to a few dozen) one can index the admissible subsets on $\lceil \log_2 N \rceil$ qubits, use an oracle that checks $\sum_{i \in S} x_i = p$, and apply $\lfloor \frac{\pi}{4}\sqrt{N} \rfloor$ iterations with the standard reflection around the uniform superposition. This works where MNRS fails on toy instances because the reflection around the mean is direct and does not depend on a QPE block that needs several qubits of precision to act correctly. What we give up is the whole point of the paper: we lose the quantum-walk speed-up and only get Grover's asymptotic behaviour. The second alternative is to build a *simplified MNRS with a compact encoding*: for a fixed instance like $(5,2)$ or $(6,3)$ one can drop the Dicke register, drop the double cover S_0 and the T copies, and initialise $|S\rangle$ directly on a single register of $\lceil \log_2 N \rceil$ qubits; the QPE block can

be replaced by an explicit reflection built from the analytical eigenvectors of the walk operator on small Johnson graphs, and the coin reduced to a single qubit when $k = 1$. The resulting circuit uses roughly 8–14 qubits and is simulable on `PyLinalg` in minutes instead of hours. This is no longer a faithful reproduction of the paper but a stripped-down variant whose purpose is to demonstrate the amplification *qualitatively* while keeping the walk structure intact.

Closing remark. The MNRS pipeline of the paper is algorithmically correct once the two localised bugs in Modifications 1 and 2 are removed (the fixed version lives in `code_updated/`), but it cannot be executed *in a meaningful way* on the toy instances $J(3, 1)$ and $J(4, 2)$: on those instances $1/\sqrt{\delta} = O(1)$, so MNRS still runs but produces no amplification window, and ℓ_s is too small for the approximate reflection to be resolved. The paper itself is explicit about this: its complexity analysis targets the cryptographic scale ($n \geq 25$ for lower T-count, $n \geq 2^{13}$ for lower T-depth) on fault-tolerant hardware, and notes that the $n \log_2 n$ qubit overhead "significantly limits the feasibility of classical simulation". The two realistic ways forward are therefore to replace MNRS by a direct Grover search on a compact vertex register, or to build a stripped-down MNRS variant on a single register and an exact reflection, keeping the walk structure but giving up the one-to-one correspondence with the paper.